

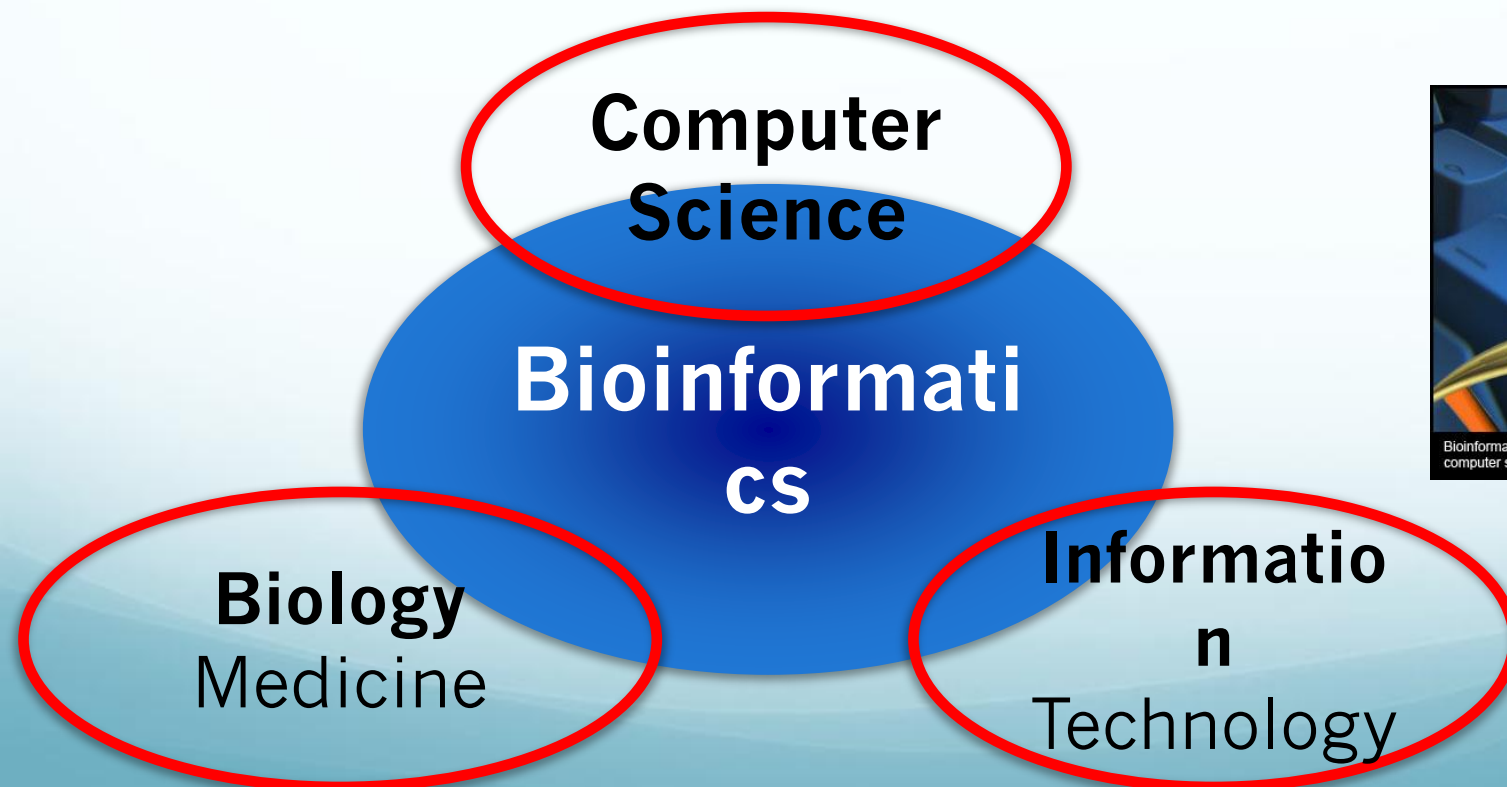
알고리즘 응용

Bioinformatics

<https://cblab.yolasite.com/>

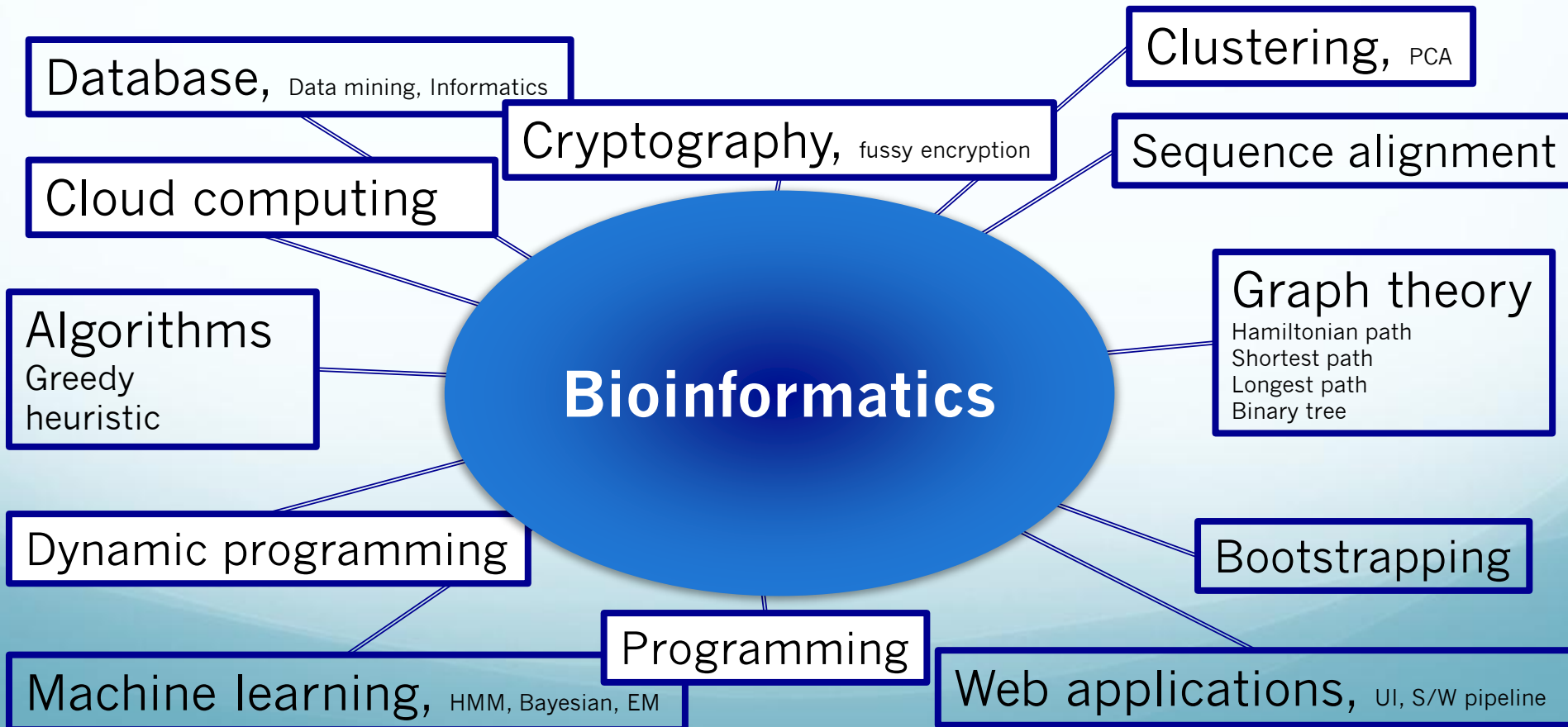
Bioinformatics?

- What is Bioinformatics?
 - 생명현상에 대한 유전적 원인을 알아내는 것.
 - Application of computer technology to manage bio big data.
 - Computer science field that deals with biological data.



Bioinformatics?

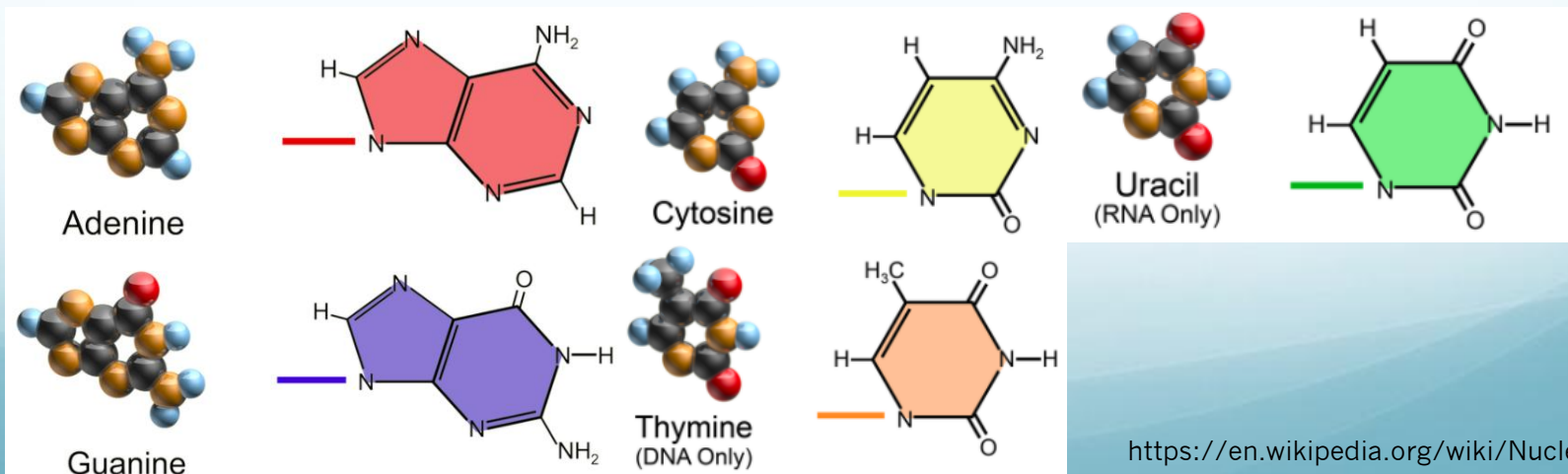
Bioinformatics closely related to many other computer science fields.



DNA Sequence

- What is the basic material for the Bioinformatics?
 - DNA , RNA, Protein
 - **DNA** sequence – A very long sequence composed of A (adenine), C (cytosine), G (guanine), T (thymine).
 - 뉴클레오타이드 (nucleotide) – DNA 사슬의 기본 구성 단위.

Structure of nucleotide



Human Genome

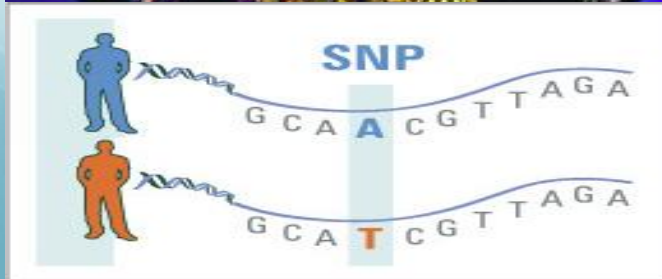
- Genome – 한생물이 가지는 모든 유전정보, ‘유전체’라고도 한다. 일반적으로 DNA로 구성된 유전정보를 지칭



Human genome sequence



- Genome sequence is composed of 4 letters; A,C,G,T.
- Human Genome sequence is length of 3 billion (30억).
- 99.9% of human genome sequence are the same. 0.1% genetics variants makes people different.
- SNP(Single Nucleotide Polymorphism, 스닙) – Single letter change in DNA.



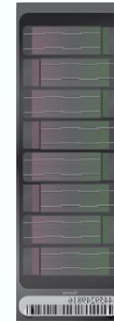
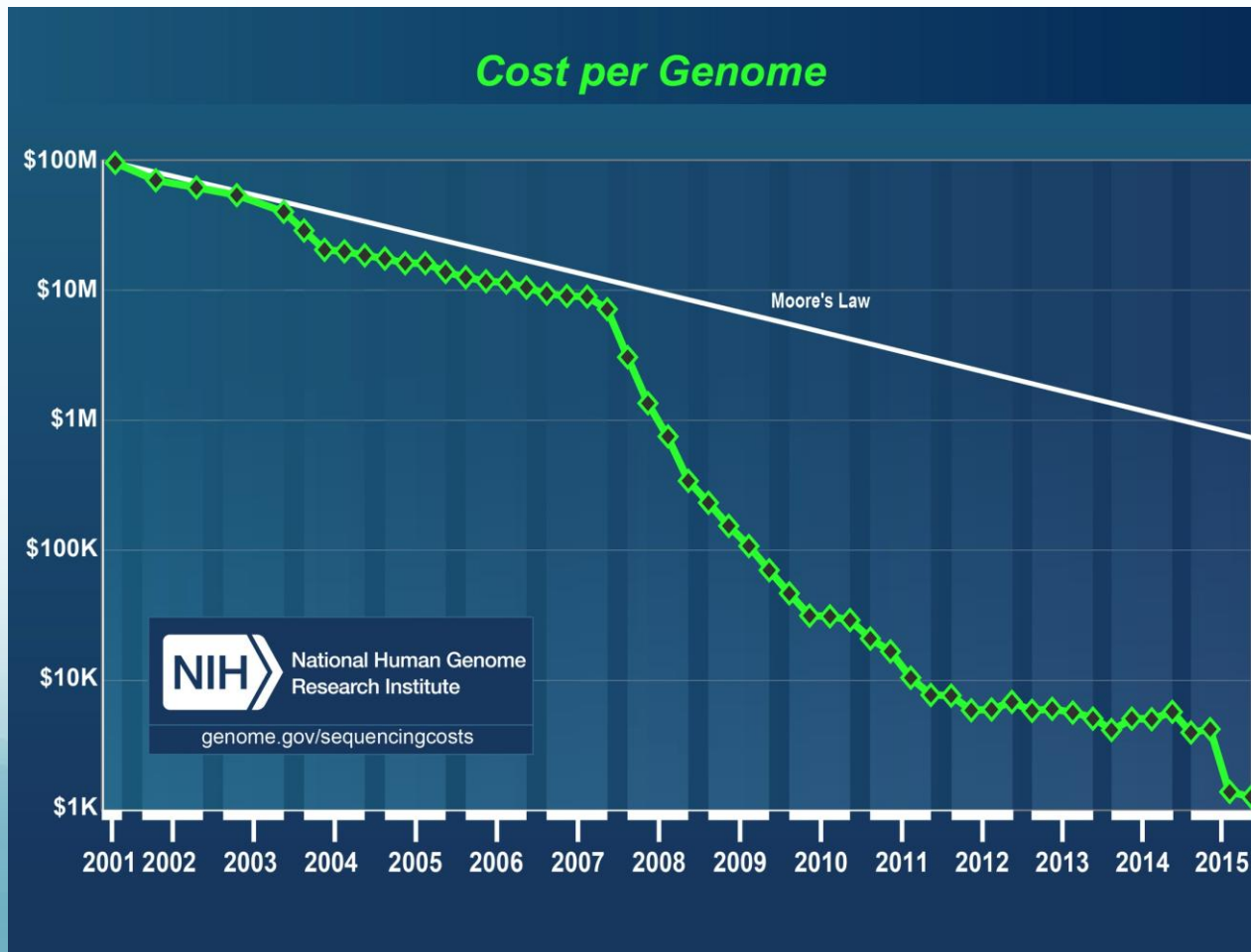
Human genome sequence

대표적 프로젝트들

1990	Human Genome Project	99% Human sequence
2002	International HapMap Project	269 Individuals , 3.9 million SNPs
2003	ENCODE (Encyclopedia of DNA Elements)	Regulatory elements
2005	Personal Genome Project	aim to get 100,000 individuals
2008	1000 Genome Project	1092 individuals
2015	GTE _x (Genotype-Tissue Expression)	Multitissues gene expression
2016	ExAC (Exome Aggregation Consortium)	High quality exome

Bio Big data

- High Throughput Technology (고속다중처리기술)



Omni2.5M
microarray



HiSeq 3000

Bio Big data

- Overwhelming volume of bio **big data** being produced. For example, Beijing genome institute produces **1 Gb/sec.**



Bio Big data - Challenges

- Accuracy (정확성)

Various unknown confounding effects exist in bio data
(Population structure, Batch effect, etc)

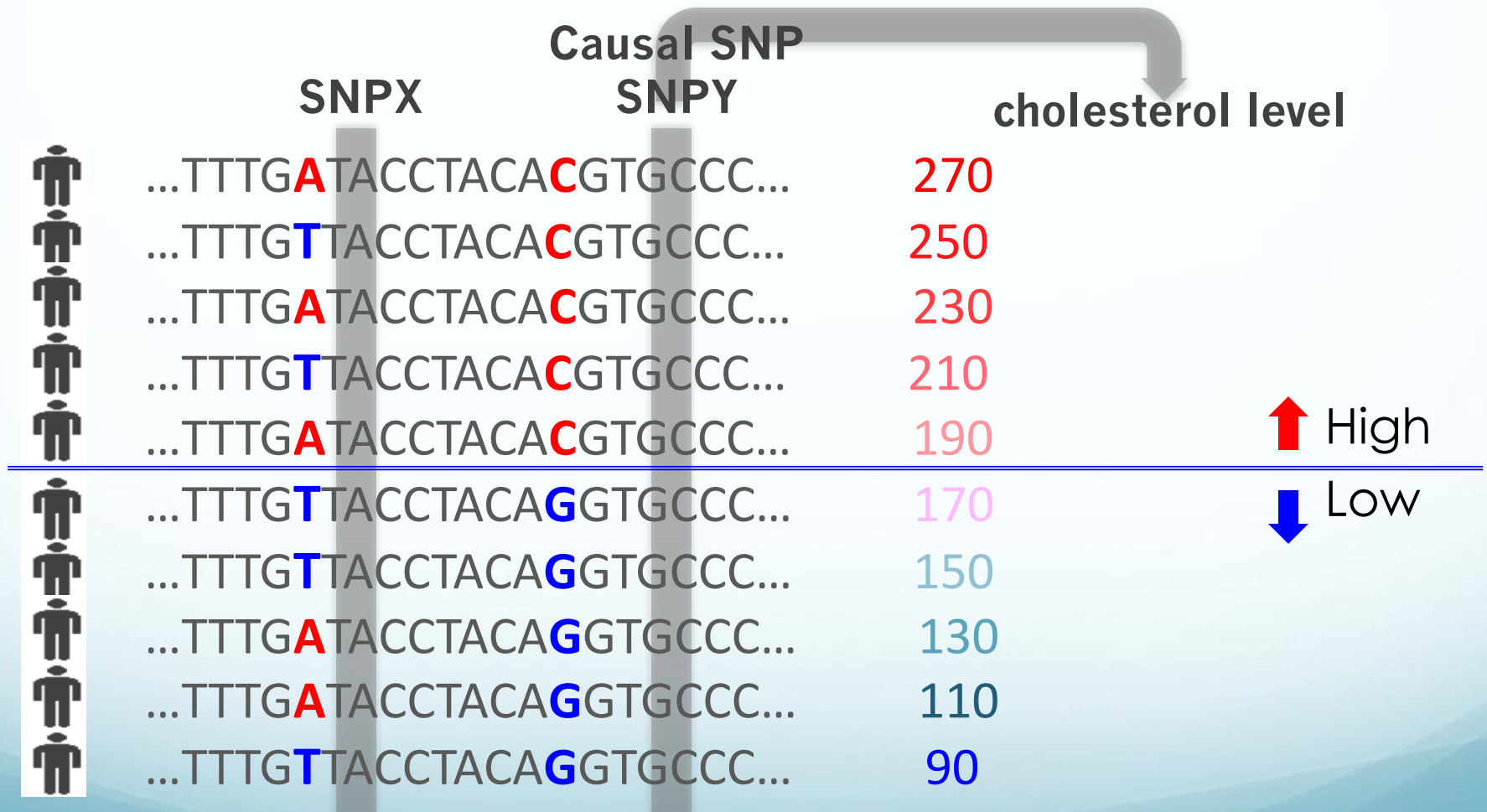
- Efficiency (효율성)

Hundreds of thousands of SNPs over thousands of
Samples



Need an **Accurate** and **Efficient**
algorithms/methods
to analyze bio big datasets !

Genetic Association Study



Genetic Association Study

환자군: 질병을 가지고 있음 대조군: 질병을 가지고 있지 않



SNP X



A : 90 % T : 10 %

대립유전자 빈도
차이가 큰가?

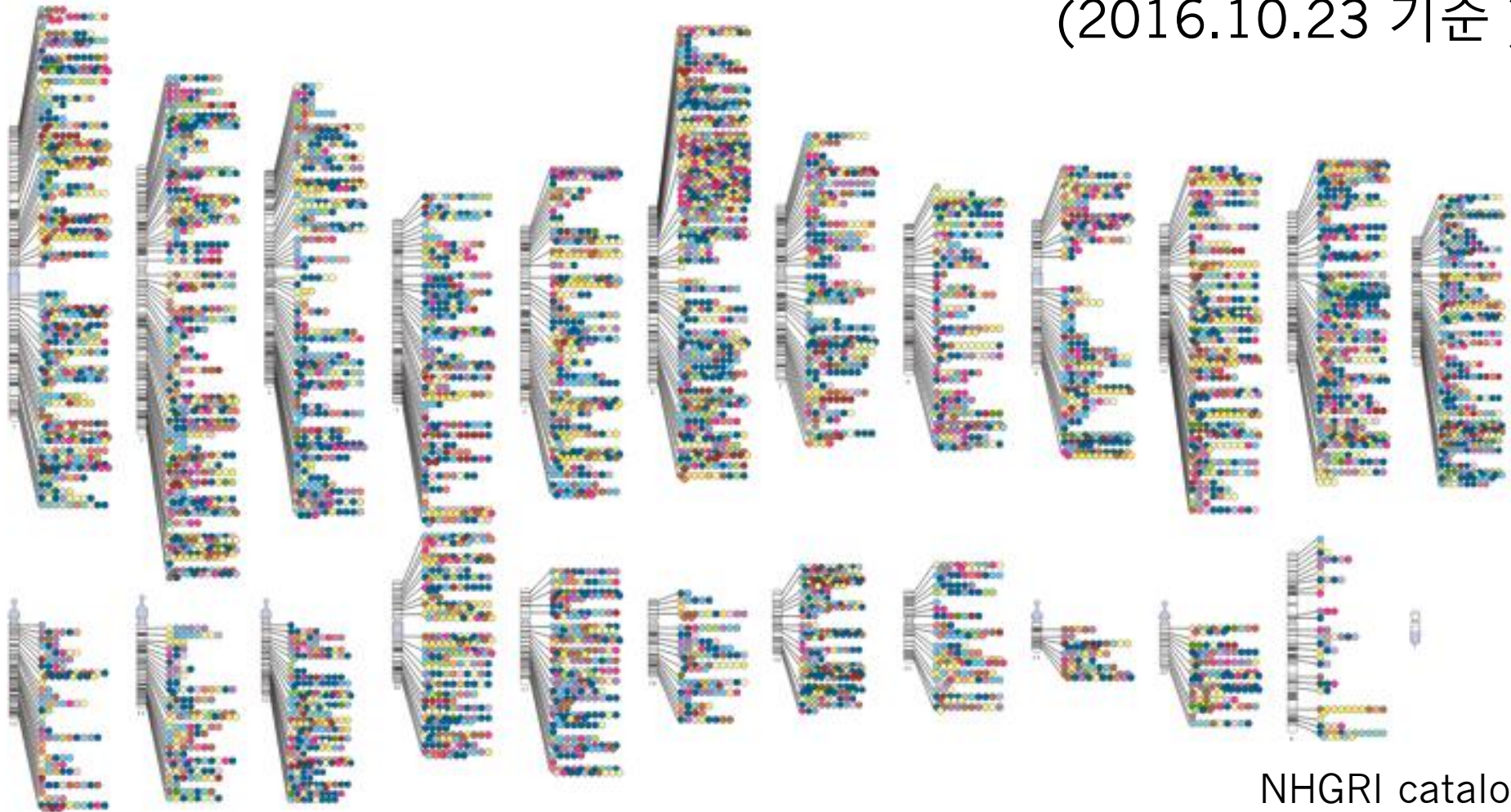
A : 60 % T : 40 %

yes

SNP X 는 질병과 연관성이 있다.

바이오 빅데이터 시대 - 연관연구 발표

SNP과 다양한 형질간의 29,323 개의 연관 발표
(2016.10.23 기준)



Short read re-sequencing Problem

Genome Sequence

AGATAACTGGGCCCC...



Sequencer

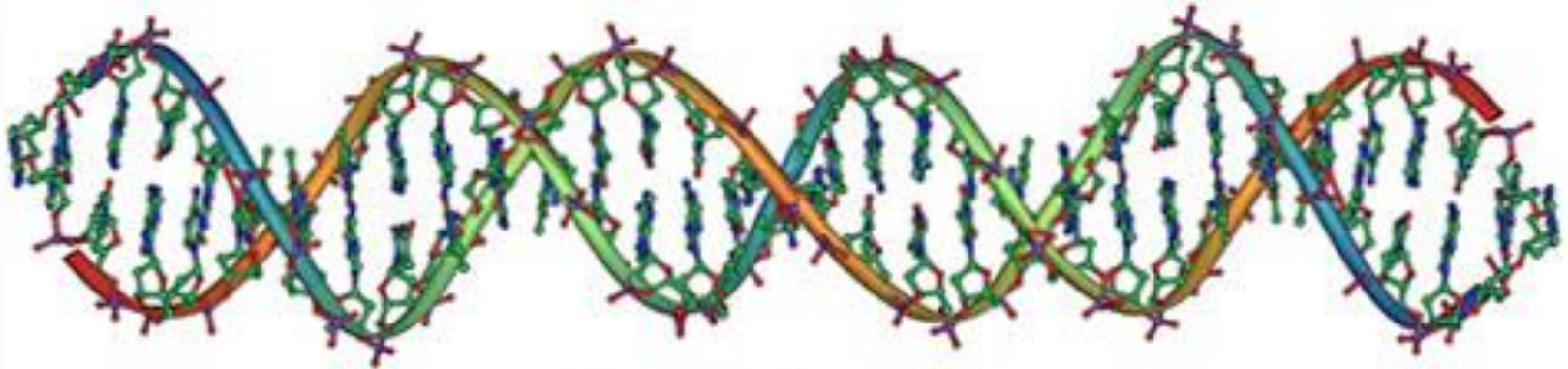
Random substring of
Genome Sequence

Short Reads

AGA
GAT
AAC
GGG
GGC
TAA
ACT

...

Short read re-sequencing Problem



Genome Sequence

AGATAACTGGGCCCCTGCGCTCAGGAGGCCTTCACCCTCTGCTCTGGGTAAAGGTAGTAGA

Fragment Reads

AGATAACTGGGCCCCTGCGCTCAGGAGGCCTTCACC

CTGGGCCCCTGCGCTCAGGAGGCCTTCACCCTCTGC

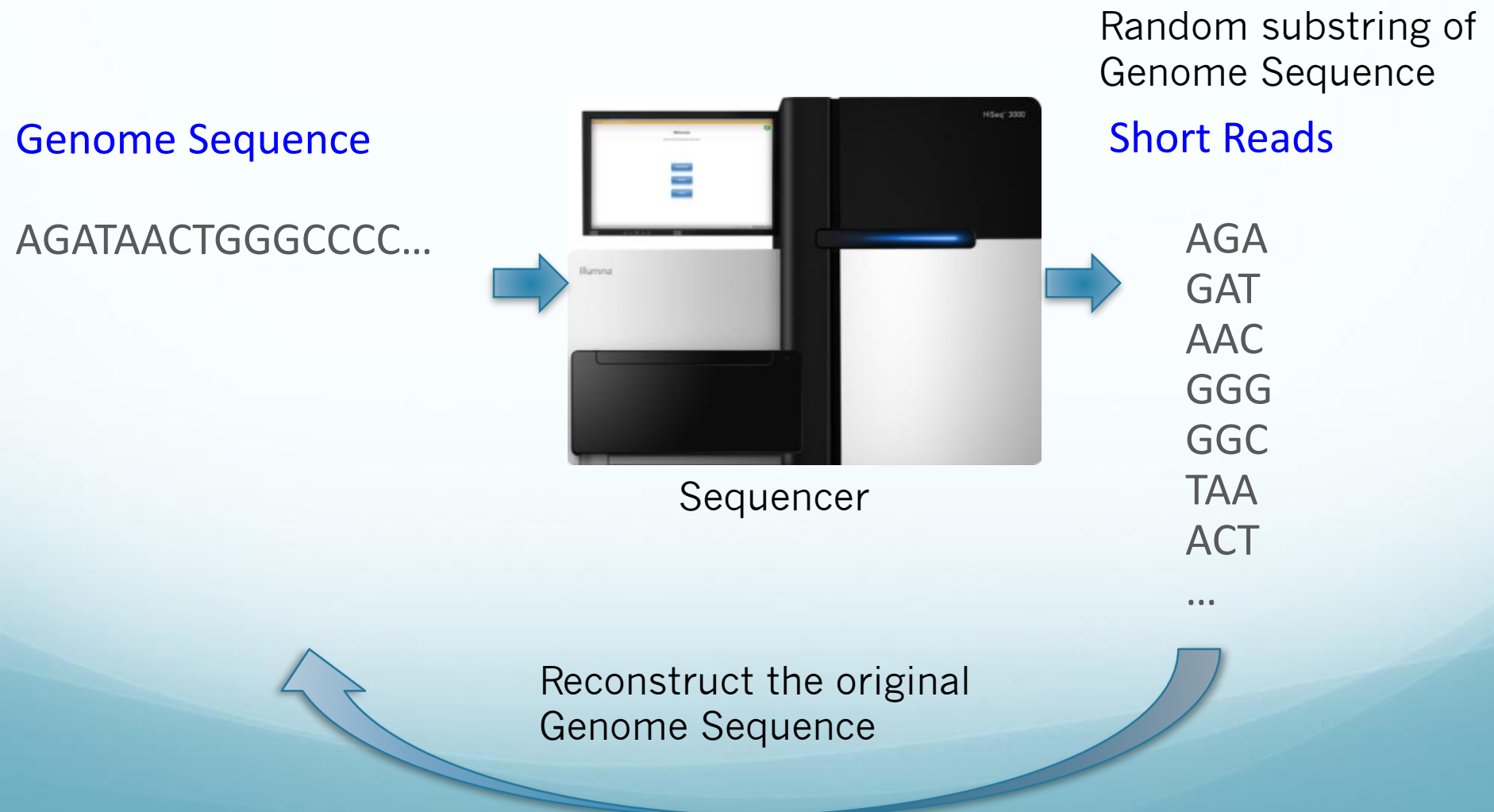
CCCCTGCGCTCAGGAGGCCTTCACCCTCTGCTCTGG

TGCGCTCAGGAGGCCTTCACCCTCTGCTCTGGGTAA

CTCAGGAGGCCTTCACCCTCTGCTCTGGGTAAAGGT

AGGCCTTCACCCTCTGCTCTGGGTAAAGGTAGTAGA

Short read re-sequencing Problem



Short read re-sequencing Problem

My Genome Sequence

AGATAACTGGGGCCCTACGGCAATATACGTACAATTTAGGA...

```
1 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
2 AG  TAACTGGGGC  CTACGGCAATATACGTACAATTTAGGA...
3 AG  TAACTGGGGCCC  TACGGCAATATACGTACAATTTAGGA...
4 AGATAACTGGGGC  CTACGGCAATATACGTACAATTTAGGA...
5 AGATAACTGGGGCCCTACG  TCAATATACGTACAATTTAGGA...
6 AGATAACTGGG  CCCTACGGCAA  TATACGTACAATTTAGGA...
7 AGATAACTGGGGC  CTACGGCAATATAC  GTACAATTTAGGA...
8 AGATAACTGGGGCCCTA  CGGCAATATACGTACAATTTAGGA...
9 AGATAACTGGGGCCCTACGGCA  ATATACGTACAATTTAGGA...
10AGATAACTGGGGCCCTACGGCAATATA  CGTACAATTTAGGA
```

...

perfect match

- How do we get someone's DNA sequence? Where are my mutations?
- Next generation sequencing - Cheap sequencing, "Short Reads"



Sequencing Technology



Illumina / Solexa
Genetic Analyzer 1G
1000 Mb/run, 35bp reads

AGAGC**A**GT**C**GAC
A**G**GTATAG**T**CTA
CATGAGAT**C**GAC
ATGAGAT**C****G**GTA
GAGC**C**GTGAGAT
C**G**ACATGATAG**C**
CAGAGC**A**GT**C**GA
CA**G**GTATAG**T**CT
ACATGAGAT**C****G**A
CATGAGAT**C****G**GT
AGAGC**C**GTGAGA
T**C****G**ACATGATAG
CCAGAGC**A**GT**C**G
ACA**G**GTATAG**T****C**
TACATGAGAT**C****G**
ACATGAGAT**C****G**G
TAGAGC**C**GTGAG

Short Read Sequencing Problem (A Computer Science Problem)

Full DNA Sequence

AGAGCAGTCGAC
AGGTATAGTCTA
CATGAGATCGAC
ATGAGATCGGTA
GAGCCGTGAGAT
CGACATGATAGC
CAGAGCAGTCGA
CAGGTATAGTCT
ACATGAGATCGA
CATGAGATCGGT
AGAGCCGTGAGA
TCGACATGATAG
CCAGAGCAGTCG
ACAGGTATAGTC
TACATGAGATCG
ACATGAGATCGG
TAGAGCCGTGAG
ATCGACATGATA
GCCAGAGCAGTC
GACAGGTATAGT
CTACATGAGATC



- Short read sequencers generate random short substrings from the DNA sequence of a certain length.

ATGAGATCGGTAGAGCCGTGAGAT
GAGCAGTCGACAGGTATAGTCTAC
AGAGCAGTCGACAGGTATAGTCTA
TGAGATCGACATGATAGCCAGAGC
TAGCCAGAGCAGTCGACAGGTATA
GATAGCCAGAGCAGTCGACAGGTA
GAGATCGACATGATAGCCAGAGCA
GCAGTCGACAGGTATAGTCTACAT
AGCAGTCGACAGGTATAGTCTACA
TCGACATGAGATCGGTAGAGCCGT
CAGTCGACAGGTATAGTCTACATG
GAGATCGACATGATAGCCAGAGCA
GTAGAGCCGTGAGATCGACATGAT

Denovo Method

- Reconstruct my genome using short reads without using reference genome.
- 예 – short read 간 겹치는 부분을 분석해 이어나감

My Genome Sequence

AGATAACTGGGGCCCTACGGCAATATACGTACAATTTAGGA...

- 1 AGATAACTG GGGCCCTACGGCAATATACGTACAATTTAGGA...
- 2 AGATAACTGGGC CCTACGGCAATATACGTACAATTTAGGA...

AGATAACTGGGC

Denovo Method

- Reconstruct my genome using short reads without using reference genome.
- 예 – short read 간 겹치는 부분을 분석해 이어나감

My Genome Sequence

AGATAACTGGGGCCCTACGGCAATATACGTACAATTTAGGA...

1 AGATAACTG GGGCCCTACGGCAATATACGTACAATTTAGGA...

2 AGATAACTGGGC CCTACGGCAATATACGTACAATTTAGGA...

AGATAACTGGGC

3 AGATAACTGGGCCC TACGGCAATATACGTACAATTTAGGA...

AGATAACTGGGCCC

Short read re-sequencing Problem

My Genome Sequence

AGATAACTGGGGCCCTACGGCAATATACGTACAATTTAGGA...

```
1 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
2 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
3 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
4 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
5 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
6 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
7 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
8 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
9 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
10 AGATAACTG  TGGGGCCCTACGGCAATATACGTACAATTTAGGA...
```

AGATAACTGGGGCCCTACGGCAATATACGTACAATTTAGGA...

Reference Genome

- Denovo method– 백지 상태에서 시작해야 하므로 시간이 많이 걸리고 정확도도 떨어짐.
- 이미 reference genome 이 알려져 있음 – SNP 위치를 제외하고는 답을 알고 있음. (단, variation은 SNP뿐이라고 가정)

My Genome Sequence

AGATAA**C**TGGGGCCCT**A**CGGCAATATA**C**GTACAATTTA**G**GA...

Reference Genome Sequence

AGATAA**A**TGGGGCCCT**G**CGGCAATATA**G**GTACAATTTA**T**GA...

↑
SNP1

↑
SNP2

↑
SNP3

↑
SNP4 ...

Reference Human Genome

- We know that my genome is very close to the Human genome.

My Genome:

TACATGAGATC**G**ACATGAGATC**G**GTAGAGCC**C**GTGAGATC

A Sequence Read:

TCGACATGAGATCGGTAGAGCCGT

The Human Genome:

TACATGAGATC**C**ACATGAGATC**T**GTAGAGC**T**GTGAGATC
TCGACATGAGATC**G**GTAGAGC**C**GT

Recovered Sequence:

TACATGAGATC**G**ACATGAGATC**G**GTAGAGCC**C**GTGAGATC

Native Method

- 가장 기본적이고 쉬운 방법 - sliding

My Genome Sequence

AGATAA**CT**GGGGCCCT**A**CGGCAATATA**CG**TACAATTTAG**G**GA...

Reference Genome Sequence

AGATAA**A**TGGGGCCCT**G**CGGCAATATA**G**GTACAATTTAT**T**GA...

- 1 AGATAA**CT**G GGGCCCTACGGCAATATACGTACAATTTAGGA...
- 2 AGATAA**CT**GGGC CCTACGGCAATATACGTACAATTTAGGA...
- 3 AGATAA**CT**GGGGCCC TACGGCAATATACGTACAATTTAGGA...
- 4 AGATAA**CT**GGGCG CCTACGGCAATATACGTACAATTTAGGA...
- 5 AGATAA**CT**GGGGCCCT**A**CGGCAATATACGTACAATTTAGGA...

Native Method

- 가장 기본적이고 쉬운 방법 – allowing a small number of mismatches, slide from the beginning.

My Genome Sequence

AGATAA**C**TGGGGCCCT**A**CGGCAATATA**C**GTACAATTTA**G**GA...

Reference Genome Sequence

AGATAA**A**TGGGGCCCT**G**CGGCAATATA**G**GTACAATTTA**T**GA...

1 AGATAA**C**TG

Native Method

- 가장 기본적이고 쉬운 방법 – allowing a small number of mismatches, slide from the beginning.

My Genome Sequence

AGATAACTGGGCCCTACGGCAATATACGTACAATTTAGGA...

Reference Genome Sequence

AGATAAATGGGCCCTGCGGCAATATAGGTACAATTTATGA...

5 TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

TGGGCCCTACG

Found a match allowing 1 mismatch !

Short Reads Difficulties

```
ATGAGATCGGTTAGAGCCGTGAGAT
GAGCAGTCGACAGGTATAGTCTAC
AGAGCAGTCGACAGGTATAGTCTA
TGAGATCGACATGATAGCCAGAGC
TAGCCAGAGCAGTCGACAGGTATA
GATAGCCAGAGCAGTCGACAGGTA
GAGATCGACATGATAGCCAGAGCA
GCAGTCGACAGGTATAGTCTACAT
AGCAGTCGACAGGTATAGTCTACA
TCGACATGAGATCGGTTAGAGCCGT
CAGTCGACAGGTATAGTCTACATG
GAGATCGACATGATAGCCAGAGCA
GTAGAGCCGTGAGATCGACATGAT
```

- We don't know where each read comes from!
- Can't identify where the mutations are!
- What do we do?

Algorithmic “Re”-sequencing Challenges

- Sequences are long! - Human Genome is 3,000,000,000 long.
- Sequencers generate many reads! - A sequencer generates over 1,000,000,000 reads.
- We need efficient algorithms to “map” each read to its location in the genome.
- There are other challenges which we are not mentioning.

Trivial Mapping Algorithm

The Human Genome:

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC

A Sequence Read:

TCGACATGAGATCGGTAGAGCCGT

- We can slide our read along the genome and count the total mismatches between the read and the genome.
- If the mismatches are below a threshold, we say that it is a match.

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC
TCGACATGAGATCGGTAGAGCCGT
↑↑ ↑↑↑ ↑↑↑↑↑ ↑↑↑↑↑ ↑↑

Total of 18 mismatches. Not below threshold. Not a match.

Trivial Mapping Algorithm

The Human Genome:

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC

A Sequence Read:

TCGACATGAGATCGGTAGAGCCGT

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC

TCGACATGAGATCGGTAGAGCCGT



Total of 23 mismatches. Not below threshold. Not a match.

Trivial Mapping Algorithm

The Human Genome:

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC

A Sequence Read:

TCGACATGAGATCGGTTAGAGCCGT

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC
TCGACATGAGATCGGTTAGAGCCGT
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑

Total of 23 mismatches. Not below threshold. Not a match.

Trivial Mapping Algorithm

The Human Genome:

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC

A Sequence Read:

TCGACATGAGATCGGTAGAGCCGT

TACATGAGATCCACATGAGATCTGTAGAGCTGTGAGATC
TCGACATGAGATCGGTAGAGCCGT



Total of 3 mismatches. Below threshold. A match!

Complexity of Trivial Algorithm

- 3,000,000,000 length genome (N)
- 300,000,000 reads to map (M)
- Reads are of length 30 (L)
- Number of mismatches allowed is 2 (D).
- Each comparison of match vs. mismatch takes $1/1,000,000$ seconds (t).
- Important: Trivial algorithm only solves problem under assumptions.
- Total Time =



Complexity of Trivial Algorithm

- 3,000,000,000 length genome (N)
- 300,000,000 reads to map (M)
- Reads are of length 30 (L)
- Number of mismatches allowed is 2 (D).
- Each comparison of match vs. mismatch takes 1/1,000,000 seconds (t).
- Important: Trivial algorithm only solves problem under assumptions.
- Total Time = $N * M * L * t = 27,000,000,000,000$ seconds
(864,164 years)

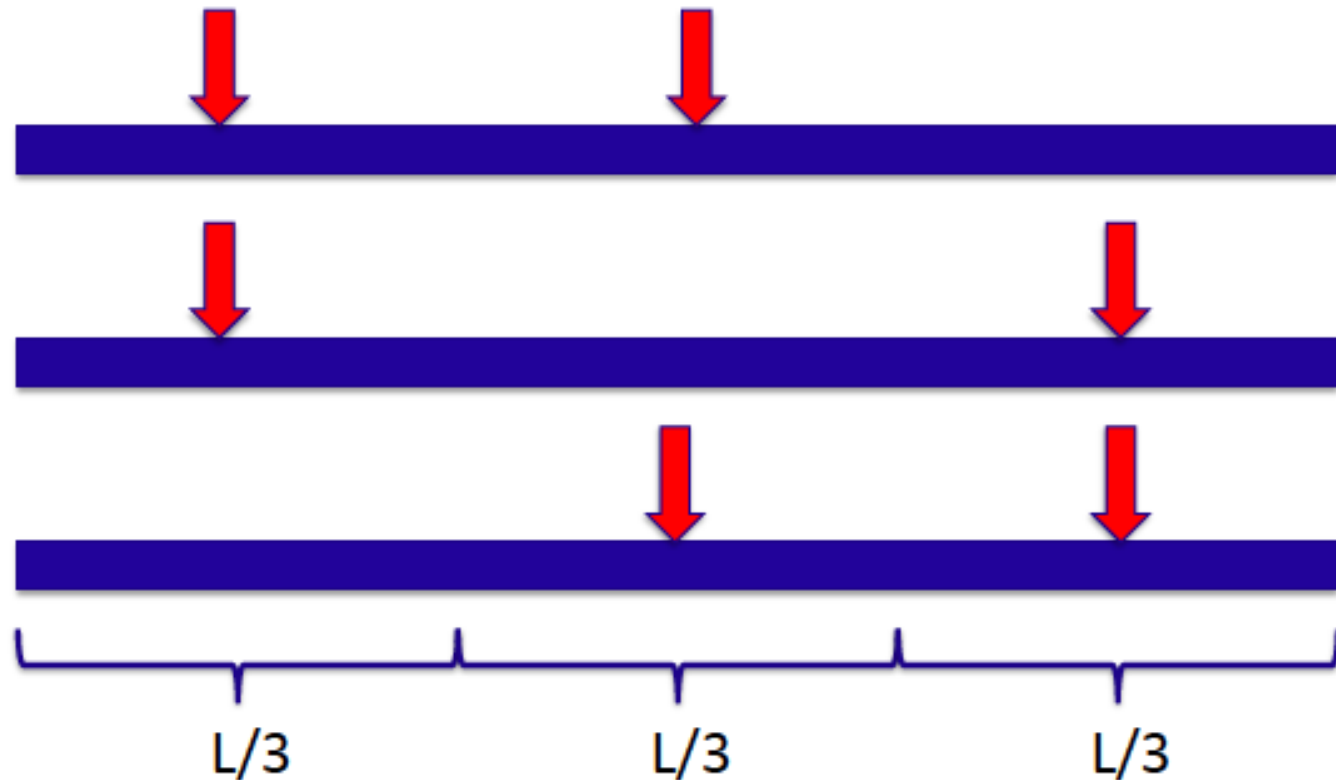
Some observations

- Most positions in the genome match very poorly.
- We are looking for only a few mismatches. (D is small)
- A substring of our read will match perfectly.

Perfect Matching Read Substrings

- For the case allowing 2 mismatches for read of

Three “worst” possible cases for placement of mutations.



- In each case, there is a perfect match of $L/3$.

Finding a perfect match of length $L/3$

- Intuition: Create an index (or phone book) for the genome.
- We can look up an entry quickly.

Sequence	Positions
AAAAAAAAAA	32453, 64543, 76335
AAAAAAAAAC	64534, 84323, 96536
AAAAAAAAAG	12352, 32534, 56346
AAAAAAAAAT	23245, 54333, 75464
AAAAAAAAACA	
AAAAAAAAACC	43523, 67543
...	
CAAAAAAAAA	32345, 65442
CAAAAAAAAAAC	34653, 67323, 76354
...	
TCGACATGAG	54234, 67344, 75423
TCGACATGAT	11213, 22323
...	
TTTTTTTTTG	64252
TTTTTTTTTT	64246, 77355, 78453

Complexity of Indexing Algorithm

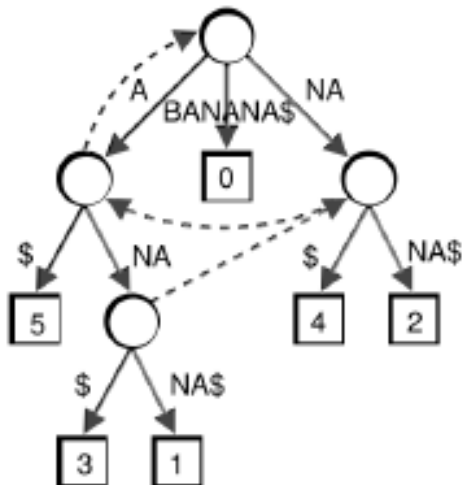


Complexity of Indexing Algorithm

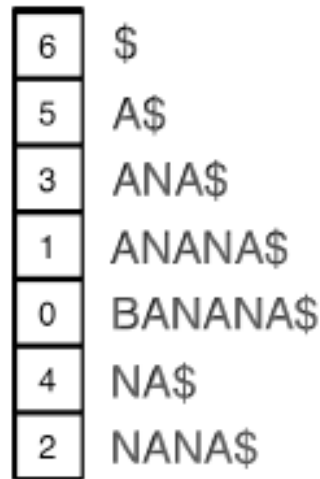
- We need to look up each third of the read in the index.
- For $L=30$, our index will contain entries of length 10. Each entry will contain on average $N/(4L/3)$ or 3,000 positions.
- For each position, we need to compute the number of mismatches.
- Our running time is $L * M * 3 * N / (4L/3) * T = 81,000,000$ seconds (937 days).
- If $L=45$, then the time is 81,000 seconds or 22.5 hours.

Indexing a genome

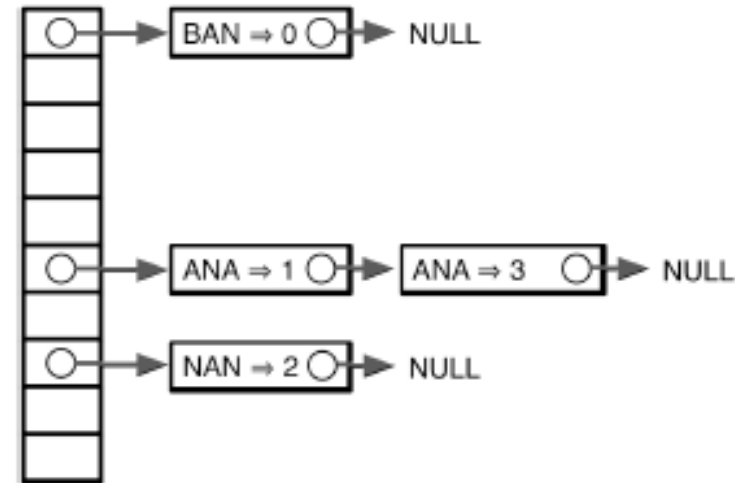
- To find exactly matching substrings, we need to build an index for the whole genome.
- Problem: The genome is BIG!
- Genome indices can be big. For human:



> 35 GBs



> 12 GBs



> 12 GBs

Burrows-Wheeler transform

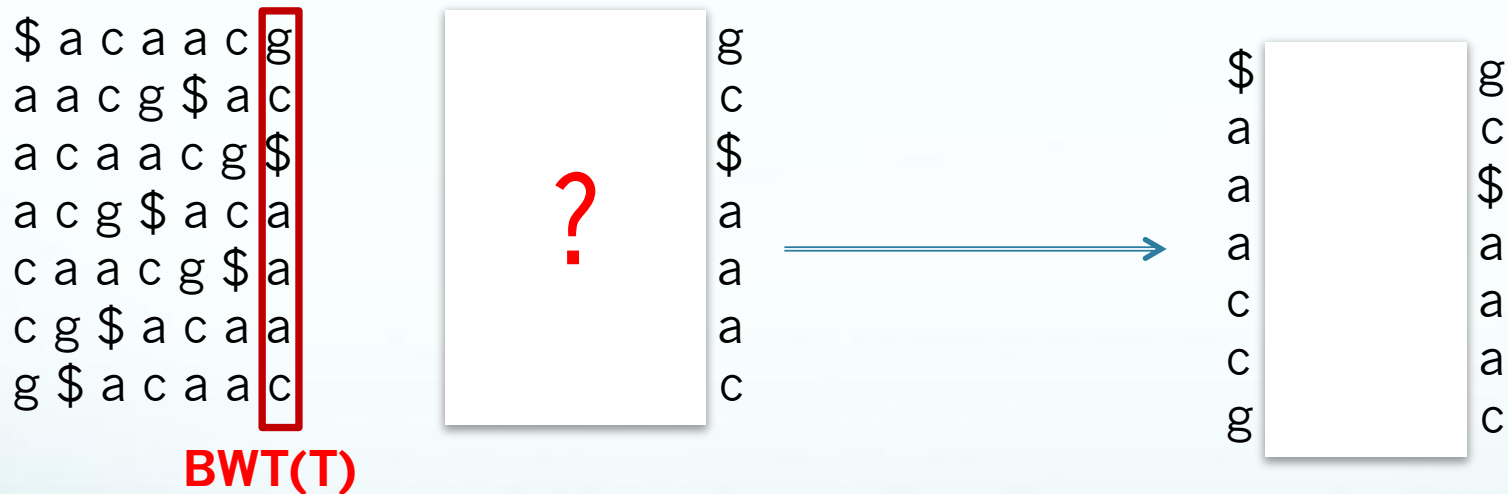
- http://en.wikipedia.org/wiki/Burrows-Wheeler_transform
- Reversible permutation used originally in compression
- $T = a c a a c g$

Burrows
Wheeler
Matrix



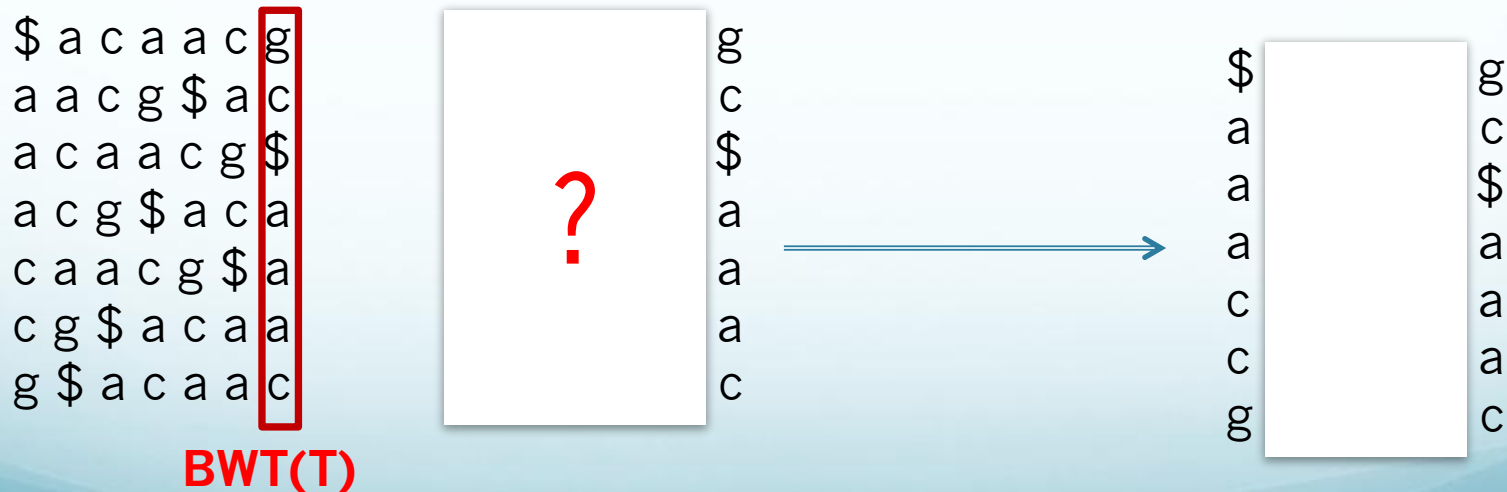
Burrows-Wheeler transform

- Can we reconstruct the first column with $BWT(T)$?



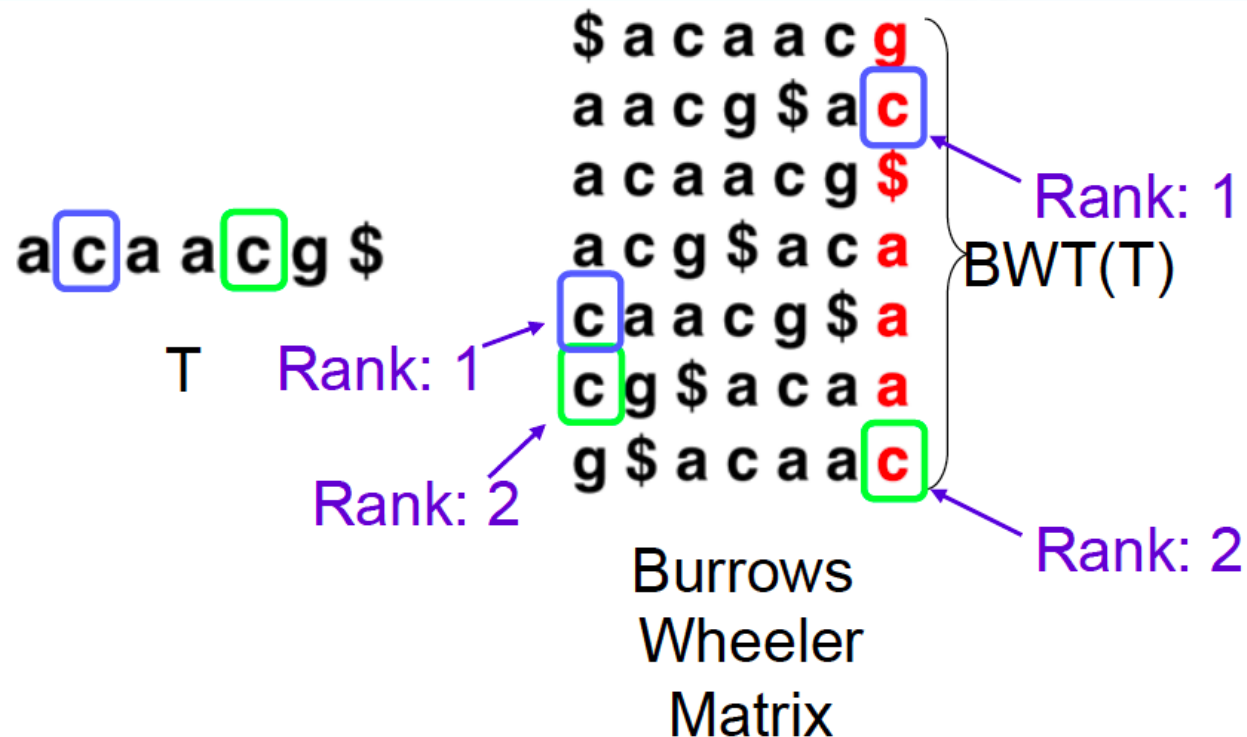
Burrows-Wheeler transform

- Can we reconstruct the first column with $BWT(T)$?
 - First column can be recovered by counting symbols in last column because it is sorted.



Burrows-Wheeler transform

- Property that makes $BWT(T)$ reversible is “LF Mapping”
 - i th occurrence of a character in Last column is same text occurrence as the i th occurrence in First column.



Burrows-Wheeler transform

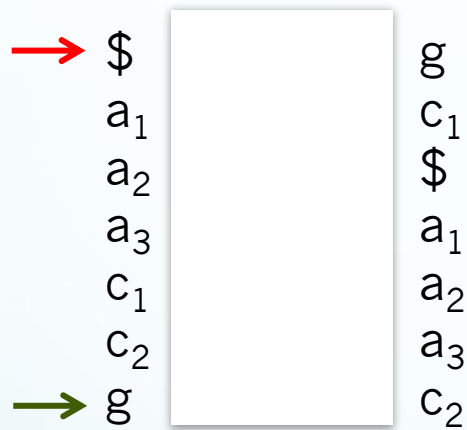
- Can we reconstruct the original sequence, $T = a\ c\ a\ a\ c\ g$, using $BTW(T)$?
- We can recover the first column as described in the previous slide.
- $T = a\ c\ a\ a\ c\ g$ reconstruct in the reverse order
 $\xleftarrow{\text{using suffix}}$

Exact Matching using BWT(T)

- Look up pattern in reverse.
 - Use 2 pointers to represent range of matches.
 - All matches will be next to each other in matrix.
 - Find first valid match for next symbol in range.
- Example BTW(T) = g c \$ a a a c, find pattern “a a c”

Exact Matching using BWT(T)

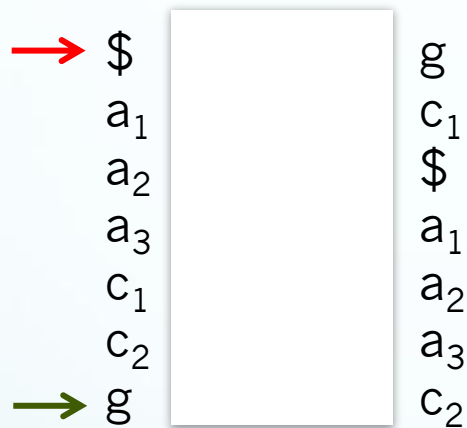
- Example $T = a\ c\ a\ a\ c\ g$, find pattern $w = a\ a\ c$



\$	g
a ₁	c ₁
a ₂	\$
a ₃	a ₁
c ₁	a ₂
c ₂	a ₃
g	c ₂

Exact Matching using BWT(T)

- Example $T = a\ c\ a\ a\ c\ g$, find pattern $w = c\ a\ a$



\$	g
a ₁	c ₁
a ₂	\$
a ₃	a ₁
c ₁	a ₂
c ₂	a ₃
g	c ₂

Exact Matching using BWT(T)

- Example $T = a\ c\ a\ a\ c\ g$, find pattern $w = g\ a\ c$

\$					g
a ₁					c ₁
a ₂					\$
a ₃					a ₁
c ₁					a ₂
c ₂					a ₃
g					c ₂

- pointers get lost.
- This way we can quickly check whether there is a match or not.

Where in sequence is the match?

- Use “walk-left” to build sequence to start
- Count number of sequences to get to the starting position.
- $w = a a c$

Where in sequence is the match?

- Use “walk-left” to build sequence to start
- Number of moves back to the starting position
- $w = c a a$

- “walk-left” to start of sequence is slow
 - Requires on average $N/2$ steps to reach start.
 - Alternate strategy: keep index of positions.
 - $T = a c a a c g$

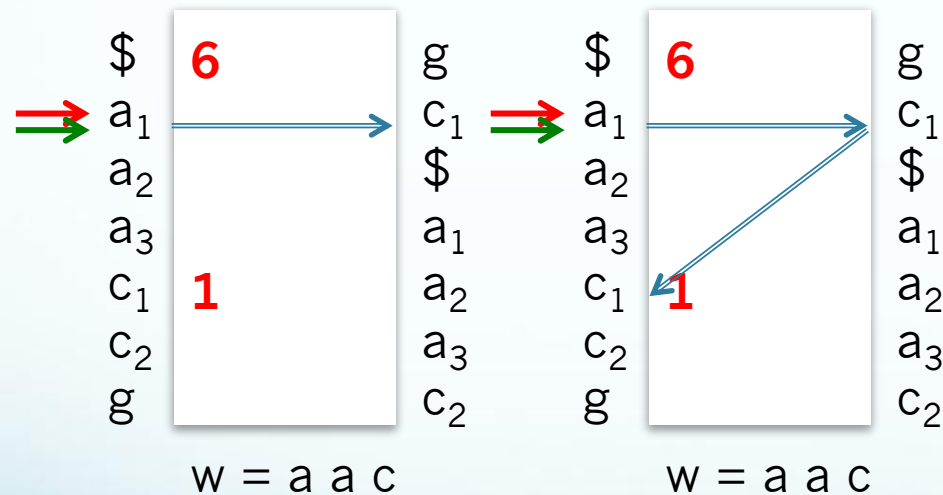


walk-left optimization

- we can save space for index
- Key Idea: Store fraction of array (sampling)
- Only store some positions and “walk-left”
- Combines two previous strategies.
- How many values to store provides defines time/space tradeoff.

walk-left optimization

- $T = a c a a c g$, $w = a a c$ is in position 2



Position =
number of moves + position in
array
For "aac" = $1 + 1 = 2$

BWT efficiency

- BWT is very fast

Table 1

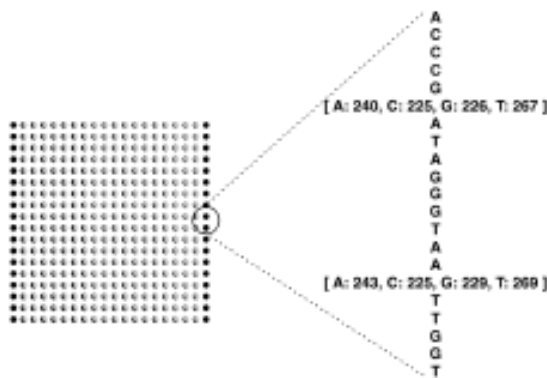
Bowtie alignment performance versus SOAP and Maq

	Platform	CPU time	Wall clock time	Reads mapped per hour (millions)	Peak virtual memory footprint (megabytes)	Bowtie speed-up	Reads aligned (%)
Bowtie -v 2	Server	15 m 7 s	15 m 41 s	33.8	1,149	-	67.4
SOAP		91 h 57 m 35 s	91 h 47 m 46 s	0.10	13,619	351×	67.3
Bowtie	PC	16 m 41 s	17 m 57 s	29.5	1,353	-	71.9
Maq		17 h 46 m 35 s	17 h 53 m 7 s	0.49	804	59.8×	74.7
Bowtie	Server	17 m 58 s	18 m 26 s	28.8	1,353	-	71.9
Maq		32 h 56 m 53 s	32 h 58 m 39 s	0.27	804	107×	74.7

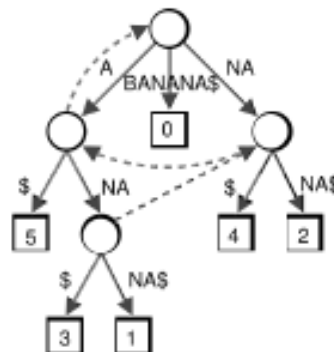
The performance and sensitivity of Bowtie v0.9.6, SOAP v1.10, and Maq v0.6.6 when aligning 8.84 M reads from the 1,000 Genome project (National Center for Biotechnology Information Short Read Archive: SRR001115) trimmed to 35 base pairs. The 'soap.contig' version of the SOAP binary was used. SOAP could not be run on the PC because SOAP's memory footprint exceeds the PC's physical memory. For the SOAP comparison, Bowtie was invoked with '-v 2' to mimic SOAP's default matching policy (which allows up to two mismatches in the alignment and disregards quality values). For the Maq comparison Bowtie is run with its default policy, which mimics Maq's default policy of allowing up to two mismatches during the first 28 bases and enforcing an overall limit of 70 on the sum of the quality values at all mismatched positions. To make Bowtie's memory footprint more comparable to Maq's, Bowtie is invoked with the '-z' option in all experiments to ensure only the forward or mirror index is resident in memory at one time. CPU, central processing unit.

BWT efficiency

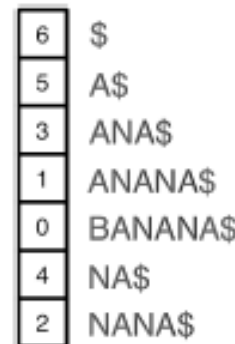
- BWT requires very small memory
- Total: $\sim 1.65x$ the size of T
- Other methods



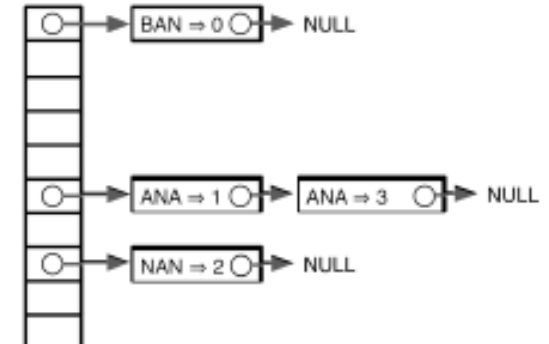
$\sim 1.65x$



$>45x$



$>15x$



$>15x$

For human: 2.2.GB

$>35GB$

$>12GB$

$>12GB$

references

- An Eulerian path approach to DNA fragment assembly, Pevzner et al, PNAS, 2011
- <http://www.homolog.us/Tutorials/index.php?p=1.1&s=1>
- http://www.cs.jhu.edu/~langmea/resources/lecture_notes/assembly_dbg.pdf

설계 프로젝트

설계 프로젝트

- Problem: Given M number of short reads of length L , reconstruct the original sequence of length N that those short reads come from.
 - 팀활동 보고서 제출
 - Construct an algorithm and implement it. 6/8기준으로 확인
 - Presentation. 6/9, 6/11 일 중 랜덤
 - 최종 보고서 제출- 6/9 10시전 까지 eclass 업로드 및 발표일에 발표전 출력물 제출 필

고려할 수 있는 요소들

- Complications that you could consider
 - N could be long (up to 3,000,000,000)
 - M could be large (typically 20million~ 200 million)
 - Length of L (typically 32~100)
 - number of mismatches in a read, D
 - There could be repeats (sequence repeats in several places)
 - single-end or paired – end reads (short read 가 일정한 gap 을 두고 두개 나오는 것)
 - denovo or reference?
 - reference genome 을 사용하지 않는 denovo sequencing 을 해도 됩니다. 이경우 장점은 새로운 species 같이 reference genome 이 없는 genome 을 sequencing 하거나 밝혀지지 않은 새로운 structural variation (insertion, repeats, inversion 등등) 을 찾아낼 수 있습니다.
 - reference genome 을 사용해도 됩니다.
 - else – 그 밖에 스스로 응용가능한 문제점들을 찾아보세요.

발표방법

- 발표시간은 4~5분, 질문 1분입니다. 다른 사람 발표에 적어도 두 번 질문해야 합니다.
- Must include the following slides
 - Introduction
 - Clearly define your problem
 - Explain your data (ex. where you got the data, how you generate the data, or etc.)
 - Input and Output
 - Benchmark - Other algorithm that you compared with your algorithm
 - Your Algorithm
 - Result
 - about your machine. (unix/mac/pc, CPU, memory size, etc)
 - time and space complexity in either big O notation or actual time/space
 - Compare with the benchmark
 - Future work
 - cons and pros of your algorithm
 - 알고리즘 개선방안 논의

평가기준 및 some tips

- 평가기준

- 난이도 – 문제의 난이도
- 정확도 – 알고리즘과 그 구현이 하고자 하는 바를 정확하게 하였는지
- 명확도 – 누구나 알아들을 수 있게 발표가 clear 한가
- 질문 두번했는지 여부
- 보고서 점수

- Tips

- 가능하면 남들과 다른 알고리즘을 사용하면 좋습니다. 같은 구현을 하였을 경우 상대 평가 될 수 있습니다.
- 발표 일주일 전까지는 대강 완성을 한다는 생각으로 미리미리 해 놓으세요. 발표 전 몇일은 발표 연습과 보고서를 쓰는데 시간이 걸릴 것 입니다.

발표시 하면 안될 것들

- 발표시간 엄수하세요. 5분지나면 더이상 발표할 수 없으므로 미리 5분 맞춰서 반드시 연습하고 오세요.
- 꼭 필요하다면 시간안걸리는 간단한 프로그램 돌려 결과를 보여주는 것은 가능하나 구현한 코드를 설명하지는 마세요. 어차피 조교랑 프로그램은 다시 검토할 것이며 코드가 아니라 보기쉽게 “알고리즘”을 설명하세요.
- 다른 논문 그대로 설명. 논문 review 발표가 아닙니다. 어떤 경우 그냥 논문을 copy & paste해서 알고리즘 설명하는 경우가 있었습니다.
- 구현 미완성 – 어려운 문제를 선택 했을 경우 구현이 잘 안될 수도 있습니다. 그래도 적어도 결과는 나와야합니다. 이유를 설명하고 어떤 때문에 성능이 원하는 만큼 안나왔다는 가능하지만 아예 결과가 도출이 안되면 안됩니다.